TRACEGUARD

Product User Guide

Scope · Test · Impact

Release readiness, in one place — for Mighty & SPL

TraceGuard reads your application's source and tells you, for every API, what a release touches, whether it actually worked in your logs, and what changed since the last release — each as a report you can share and sign off from.

Audience: Release, Development & Test leads • **Applies to:** TraceGuard desktop & server • **Classification:** Internal use

IN THIS GUIDE

Contents

1	What TraceGuard does for you	The three questions it answers
2	Before you begin	Opening the tool and choosing your source
3	Key terms	A short, plain-language glossary
4	Release Scope	See what an API touches
5	Release Test	Prove the release actually worked
6	Release Impact	Know what changed between releases
7	Reports you can share	Every deliverable, at a glance
8	Configuration essentials	One-time setup in ⚙ Config
9	Tips & troubleshooting	Quick answers to common questions

SECTION 1

What TraceGuard does for you

TraceGuard is a release-readiness tool. It points at your application's source — it never needs the application running — and turns hours of manual code-reading and log-hunting into three clear, shareable answers.

Scope

For any API, see the exact path it follows — its route, every backend it calls, and the service version of each — as an interactive diagram and a catalog.

Test

From your logs (or a Splunk export), confirm which APIs a release actually exercised, whether each passed end-to-end, and whether it hit the right backend version.

Impact

Compare two releases: added, removed and modified routes, backend version bumps and request-payload changes — with the author of each change.

Each answer lives on its own tab and **exports as a shareable, sectioned PDF** (and the test coverage as an Excel workbook), so your release, development and test teams can review and sign off from one document.

> **Two applications, one tool.** TraceGuard serves **Mighty** and **SPL** from a single entry point. They share the tool but are traced and analysed separately, so results never mix between applications.

SECTION 2

Before you begin

Getting to your first result takes three choices. Nothing needs to be installed by you beyond the tool itself.

1. **Open TraceGuard**
   Launch the app. It opens automatically in your browser at `http://localhost:8080`. If the browser doesn't open, type that address in yourself.

2. **Choose the application**
   Pick
   Mighty
   or
   SPL
   . Your choice — and the source below — is remembered per application.

3. **Point it at a source**
   Choose a
   Local path
   (a folder on your machine) or a
   Bitbucket branch
   . This is the application source TraceGuard reads; the same setting is shared by all three tabs.

> **One machine requirement.** Only the **Release Impact** tab needs `git` available on the machine running TraceGuard, for its change-detection. Release **Scope** and **Test** work fully without it.

A **light / dark theme** toggle sits in the top corner. Settings such as your Splunk address and uploaded reference reports are entered once in `⚙ Config` (see Section 8) and are kept safely across tool updates.

SECTION 3

Key terms

A few words appear throughout the tool. Here's what each means, in plain language.

Term	What it means
API	A single service endpoint a client app calls — identified by its operation name, e.g. *getPayeeList*.
Route	The internal processing path an API runs through inside the application.
Backend	A downstream host service the route calls to do the real work.
Service version	The version of a backend contract the route is calling — TraceGuard checks the release used the one you expect.
Client / release version	The app release under review, e.g. `9.14`. TraceGuard scopes everything to it.
Module	A distinct part of the release, traced on its own. A release may span several modules; TraceGuard can analyse them together.
BAU regression	Live behaviour changes: a production ("business-as-usual") route, or code it runs, was edited. Regression-test the current app.
Backward compatibility (BC)	Older app versions still call what changed. An old client against the new build must pass.
New app only	Versioned to the release: a new route, new fields or a backend bump on the new route. The old app never sees it.
Shared code	One Java class serves several route versions. Test every version that runs the changed method: this release, live, upcoming.

SECTION 4 — THE FIRST TAB

Release Scope · See what an API touches

Understand, at a glance, the full footprint of an API or an entire release — before a line of testing begins.

How to use it

1. **Leave the inputs empty for the full catalog**
   Every discovered API appears, grouped by the release its route resolves to.

2. **Enter a client version to scope it**
   Type a release (e.g. `9.4`) to see only the APIs that release impacts. Enter `N/A` or `latest` to resolve each API to its newest available route.

3. **Explore the flow graph**
   Read the API's path visually — colour-coded roles, the entry route, and each backend shown with its `svc vX.Y` version chip. Zoom, search and fit as needed.

What you can share

PDF catalog report

The whole release's impacted-API catalog — each API with its resolved route, flow and backends. Always the full catalog, so it can't under-report a release.

Graph image (PNG)

Export the flow diagram itself to drop into a slide or a review pack.

SECTION 5 — THE SECOND TAB

Release Test · Prove the release worked

Move from "the code looks right" to "the release ran correctly" — verified against your real logs, end to end.

How to use it

1. **Select the APIs to analyse**
   Choose one, several, or all. You can also tick the changed routes or backends to pull in the APIs a change impacts.

2. **Get the logs**
   Copy the generated **Splunk query** and run it, then paste the job's **Job SID** to build ready-made download links — *no export permission needed*. Or simply **upload an output log**. The file type is detected automatically.

3. **Verify with logs**
   TraceGuard correlates the logs against the trace and shows a readiness verdict.

What you'll see

- A **status donut** and clickable **filter chips** (All / Issues / per status), worst-first.
- A **per-API verdict** — pass/fail end-to-end, response latency and attempt counts.
- An expandable **backend breakdown**, each backend carrying a **service-version chip**: a green tick when the logged version matches the expected one, and a clear flag when it doesn't — even if the call otherwise succeeded.

> **Non-standard success codes?** Some backends report their outcome under a different field. The `🚦 Rules` editor lets you tell TraceGuard, per backend, which field to read and what counts as success — or to neutrally *skip* a backend so it never fails an API.

What you can share

PDF verification report

The full readiness picture — verdicts, coverage and backend detail — for sign-off.

Release Test — Capability Matrix Excel

Hands the testing team the **steps to test** each impacted API next to that API's verdict.

The Capability Matrix export. When you run several **modules**, the workbook gives you **one sheet per module** (the tab is named after the module) plus a leading **ALL** tab that lists every module's rows together — with a **Module** column and the test verdict — so the whole release reads top to bottom without switching tabs. APIs with no matching capability are listed in an *Unmapped* section. This export needs the two VAL reference reports attached once in `⚙ Config` (see Section 8).

SECTION 6 — THE THIRD TAB

Release Impact · Know what changed

See exactly what a release changed versus the version before it — and how much risk each change carries.

How to use it

1. **Enter the target client version**
   For example `9.18`, then choose
   Compare
   . TraceGuard diffs each API against its immediate-lower version.

2. **Optionally add the commit / app version**
   Enter it (e.g. `19.18.0`) to also detect, from source history,
   shared code changes
   and
   in-place edits to production (BAU) routes
   — the actual change, attributed to its author.

Summary view — the test plan

Written for managers: one line says how much the release touches, then four buckets. Each row gives the API, what changed in one phrase, what to test, and who tests it. APIs are grouped by business feature (payment, manage, …) and the highest risk comes first.

Bucket	Question it answers	Who tests
BAU regression	What changes for customers on the current app? A live route was edited, or code it runs changed.	BAU team
Backward compatibility	Will older app versions still work against what changed? Lists the versions that still call it.	BAU team
New app only	What is confined to the new version? New APIs, new fields or backend bumps on the new route — the old app is unaffected.	New-app team
Shared code	Which route versions run a changed class? Listed as *This release / Live now / Upcoming*.	Both teams

The four definitions are repeated under the plan, so a reader new to the tool never has to ask what BAU or BC means.

Detailed view — the evidence

- One card per API, in the same words: a **who-tests** chip, the **risk level**, a **BC** marker when it applies, and the test result once a log is attached.

- The card's first line is the verdict, e.g. *BAU regression · channel: MOBILE → APP*; expand it for the **test focus** and the full route, class and payload diffs.
- Live-route edits lead, because they carry the most risk; everything else is grouped by release version.

What you can share

Summary PDF

The test plan on one page — the four counts, who tests what, and the definitions — for a go / no-go conversation.

Detailed PDF

Every change in full, grouped by version and feature, for the development and test teams.

SECTION 7

Reports you can share

Every tab produces a deliverable you can circulate. Here they are in one place.

From	Deliverable	Format	Best for
Release Scope	Impacted-API catalog report	PDF	Understanding a release's footprint
Release Scope	Flow diagram	PNG	Slides and review packs
Release Test	Verification report	PDF	Release readiness sign-off
Release Test	Capability Matrix (per-module + ALL tab)	Excel	Handing test steps + verdicts to the test team
Release Impact	Summary report (test plan)	PDF	Go / no-go decisions — who tests what
Release Impact	Detailed report	PDF	Full change review by dev & test

Consolidated across modules. When a release spans several modules, the PDFs and the Capability Matrix consolidate every module into a single document — one pack per release, not one per module.

SECTION 8

Configuration essentials

A handful of one-time settings, all under `⚙ Config`. Your configuration is kept safely across tool updates.

Setting	Why you set it
Splunk base URL	Enables the Release Test download links that open Splunk results in your own browser. Set once.
VAL reports (Interface Spec + Capability Matrix)	Attach both Excel reference files once to unlock the **Release Test — Capability Matrix** export, which maps each impacted API to how it should be tested.
Host response-code rules (🚦)	Teach TraceGuard how specific backends report success, so verdicts stay accurate.

> **Usually pre-loaded.** Your module lists and host rules typically ship with the tool, so many teams never need to touch this screen beyond the Splunk address and VAL reports.

SECTION 9

Tips & troubleshooting

If…	Then
The screen doesn't seem to update	TraceGuard refreshes only when the source changed — press the Load / Compare button again.
The browser didn't open on launch	Go to `http://localhost:8080` manually. If another program uses that address, close it.
Release Impact shows no code changes	Confirm `git` is available on the machine running TraceGuard, and that the commit / app version was entered.
The Capability Matrix export is greyed out or empty	Attach both VAL reports in `⚙ Config` — the export needs the Interface Spec and the Capability Matrix.
A backend shows as failed but you expected success	It may report success under a non-standard code — add a rule in the `🚦 Rules` editor.
You can run a Splunk search but not export it	Paste the **Job SID** and result count in Release Test; TraceGuard builds download links that use your own Splunk session — no export right needed.

> **In one line.** Point TraceGuard at your source, pick the application, and let each tab answer one question — **what a release touches, whether it worked, and what changed** — then share the report.